

Instituto Tecnológico de Costa Rica
Escuela de Computación
Taller de Programación — Grupo 5

Documentación del Proyecto

S U D O K U — Versión 2

Programa 3: Mantenimiento de Software

Aldrickson Díaz Tijerino
Carné: 2026800632

Profesor: William Mata Rodríguez
Semestre I – 2026

Contenidos

1. Enunciado del proyecto
 - 1.1. Definición del proyecto
 - 1.2. Objetivos del proyecto
 - 1.3. Resumen de requerimientos
2. Temas investigados
 - 2.1. Software de control de versiones y colaboración
 - 2.2. Git y GitHub: diferencia entre ellos
 - 2.3. Tres paquetes de software de control de versiones
 - 2.4. Mejores prácticas con control de versiones
3. Solución — detalle de mejoras implementadas
 - 3.1. Mejora 1: Sistema de cuentas
 - 3.2. Mejora 2: Generar partidas automáticamente
 - 3.3. Mejora 3: Panel de elementos definido por el jugador
 - 3.4. Mejora 4: Juegos multinivel
 - 3.5. Mejora 5: Top X con ABB y pickle
 - 3.6. Mejora 6: Botón PISTA
4. Conclusiones
5. Estadística de tiempos
6. Lista de revisión del proyecto

1. Enunciado del proyecto

1.1. Definición del proyecto

Este proyecto corresponde a la etapa de **mantenimiento de software** sobre el Sudoku del Programa 2. En la ingeniería de software el ciclo de vida (SDLC) incluye una fase de mantenimiento cuyo objetivo es garantizar que el software siga vigente mediante mejoras, corrección de errores y adaptación al entorno. Se implementaron seis mejoras: sistema de cuentas con autenticación por correo, generación automática de tableros, elementos personalizados, modo multinivel, bitácora con Árbol Binario de Búsqueda en pickle, y un botón PISTA.

1.2. Objetivos del proyecto

- — Realizar mantenimiento de software mediante mejoras al Sudoku del Programa 2.
- — Implementar el Árbol Binario de Búsqueda (ABB) con inserción y recorrido recursivos.
- — Aplicar POO: clases Partida, NodoABB y ABB con atributos y métodos.
- — Usar GitHub con commits desde el inicio según Conventional Commits.
- — Fomentar investigación: control de versiones, Git y GitHub.

1.3. Resumen de requerimientos

#	Mejora	Puntos
1	Sistema de cuentas para acceder a la aplicación	20
2	Generar partidas automáticamente en tiempo real	10
3	Panel de elementos — tercer grupo definido por el jugador	10
4	Juegos multinivel (Fácil → Intermedio → Difícil → ciclo)	10
5	Cambios para el manejo del Top X con ABB y pickle	40
6	Funcionalidad definida por el programador (botón PISTA)	10
	TOTAL	100

2. Temas investigados

2.1. Software de control de versiones y colaboración

Marco teórico

El software de control de versiones (VCS) registra los cambios sobre archivos a lo largo del tiempo, permitiendo recuperar versiones anteriores, comparar diferencias y revertir errores. Es una práctica fundamental en la ingeniería de software moderna.

Existen dos categorías: sistemas **centralizados** (SVN, CVS) con servidor único, y sistemas **distribuidos** (Git, Mercurial) donde cada desarrollador tiene copia completa del repositorio.

Importancia en el proyecto

- Trazabilidad: cada cambio registrado con autor, fecha y descripción.
- Recuperación: volver a cualquier versión anterior ante errores.
- Colaboración: múltiples desarrolladores sin conflictos.
- Auditoría: revisión de quién cambió qué y cuándo.

Cómo se usó

Se usó Git localmente y GitHub (repositorio sudokuv2, cuenta AldricksonYesua). Commits con Conventional Commits (feat:, fix:, docs:) a lo largo del desarrollo.

Campo	Detalle
Objetivo del uso	Investigar el concepto de VCS y su importancia.
Herramienta utilizada	Claude (Anthropic)
Prompt o pregunta	¿A qué se refiere el software de control de versiones y cuál es su importancia en el desarrollo de software?
Respuesta	La IA explicó VCS centralizados vs distribuidos, componentes y rol en el SDLC. Verificado con git-scm.com.
¿Cómo usó o adaptó la respuesta?	Base para el marco teórico. Adaptada al contexto del proyecto con ejemplos de GitHub.
Reflexión crítica	Información precisa y organizada. Sin errores detectados.

2.2. Git y GitHub: diferencia entre ellos

Marco teórico

Git es un sistema de control de versiones distribuido (Linus Torvalds, 2005). Programa local, sin internet para operaciones locales. **GitHub** es plataforma web (2008, Microsoft) que aloja repositorios Git y agrega colaboración: pull requests, issues, Actions CI/CD.

Característica	Git	GitHub
Naturaleza	Software local	Plataforma web en la nube
Requiere internet	No	Sí
Función principal	Control de versiones local	Alojamiento y colaboración
Alternativas	Mercurial, SVN	GitLab, Bitbucket

Campo	Detalle
Objetivo del uso	Clarificar la diferencia entre Git y GitHub.
Herramienta utilizada	Claude (Anthropic)
Prompt o pregunta	¿Cuál es la diferencia entre Git y GitHub? ¿Son lo mismo?
Respuesta	La IA explicó Git como VCS local y GitHub como plataforma web. GitHub no es necesario para usar Git.
¿Cómo usó o adaptó la respuesta?	Tabla comparativa. Se añadieron alternativas.
Reflexión crítica	Clara y verificable. Útil para articular con precisión ambos conceptos.

2.3. Tres paquetes de software de control de versiones

Marco teórico

Git (Distribuido, 2005)

Más usado en el mundo. Copia completa en cada desarrollador. Base de GitHub/GitLab. Usado en este proyecto.

Subversion (SVN) (Centralizado, 2000)

Apache. Historial en servidor central. Usado en organizaciones con infraestructura heredada.

Mercurial (Hg) (Distribuido, 2005)

Similar a Git, interfaz más simple. Usado por Facebook y Mozilla. Menos popular actualmente.

Campo	Detalle
Objetivo del uso	Investigar tres paquetes de VCS.
Herramienta utilizada	Claude (Anthropic)
Prompt o pregunta	Menciona y explica brevemente tres paquetes de software de control de versiones populares.
Respuesta	Describió Git, SVN y Mercurial con características y casos de uso. Verificado con fuentes oficiales.
¿Cómo usó o adaptó la respuesta?	Organizado en ficha por paquete. Añadido contexto de uso en el proyecto.
Reflexión crítica	Información correcta. Comparación Git vs SVN útil para entender evolución de VCS.

2.4. Mejores prácticas con control de versiones

Marco teórico

- **Commits atómicos y frecuentes:** Cada commit representa un cambio lógico completo. Preferir muchos commits pequeños.
- **Mensajes descriptivos (Conventional Commits):** feat: nueva función, fix: corrección, docs: documentación.
- **Uso de ramas (branching):** main siempre funcional. Nuevas funciones en ramas separadas.
- **Archivos .gitignore:** Excluir binarios, credenciales y archivos generados automáticamente.
- **Etiquetas (tags):** Marcar versiones con v1.0, v2.0 para identificar entregas.

Campo	Detalle
Objetivo del uso	Documentar mejores prácticas con Git/GitHub.
Herramienta utilizada	Claude (Anthropic)
Prompt o pregunta	¿Cuáles son las mejores prácticas al usar Git y GitHub en ingeniería de software?
Respuesta	Describió commits atómicos, mensajes, branching, .gitignore y tags. Menciona Conventional Commits.
¿Cómo usó o adaptó la respuesta?	Las cinco prácticas más relevantes para proyecto individual estudiantil.
Reflexión crítica	Verificables en literatura IS. Conventional Commits útil para historial legible.

3. Solución — detalle de mejoras implementadas

3.1. Mejora 1: Sistema de cuentas para acceder a la aplicación (20 pts)

Descripción

Se implementó autenticación de usuarios antes del juego. Gestiona registro de nuevos usuarios e inicio de sesión con código de 6 dígitos enviado por correo electrónico real.

Implementación

- Archivo **usuarios.json**: lista con id, correo, codigo_ingreso (SHA-256), nombre, fecha_creacion.
- Función **mostrar_login(root)**: ventana con tres pantallas internas intercambiables (correo, registro, código).
- Función **hashear_codigo()**: encripta con `hashlib.sha256().hexdigest()`.
- Función **enviar_codigo_correo()**: `smtplib.SMTP_SSL` con Gmail puerto 465 y App Password.
- El nombre del jugador ya no se pide en el juego; viene de `self.usuario["nombre"]`.

Cambios en el programa

Imports agregados: `hashlib`, `smtplib`, `MIMEText`. Constantes: `ARCHIVO_USUARIOS`, `CORREO_REMITENTE`, `CONTRASENA_APP`. Funciones nuevas: `cargar_usuarios()`, `guardar_usuarios()`, `hashear_codigo()`, `enviar_codigo_correo()`, `mostrar_login()`. `SudokuApp` recibe parámetro `usuario`. `entry_jugador` reemplazado por `Label`.

3.2. Mejora 2: Generar partidas automáticamente (10 pts)

Descripción

Se eliminó sudoku2026partidas.json. El programa genera tableros válidos con solución única en tiempo real, con anti-repetición de hasta 50 partidas por nivel en memoria.

Implementación

- **_gen_llenar(tablero)**: backtracking recursivo que llena un tablero 9×9 completamente válido.
- **_gen_contar(tablero)**: cuenta soluciones con MRV. Se detiene en 2 para verificar unicidad.
- **_generar_tablero(vacios_obj)**: genera solución completa, elimina celdas verificando `_gen_contar()==1` en cada remoción y aplica una segunda verificación `_gen_contar(puzzle)==1` antes de retornar. La función `_tiene_solucion_simple` fue eliminada y reemplazada completamente por `_gen_contar`.
- **_historial_tableros**: dict con tres deque(maxlen=50) uno por nivel, guardando huellas de tableros.
- Celdas vacías: Fácil=36, Intermedio=47, Difícil=52.

Cambios en el programa

Eliminados: ARCHIVO_PARTIDAS, obtener_partida_aleatoria(), inicializar_partidas(). Agregados: _VACIOS_POR_NIVEL, _historial_tableros, _tablero_a_huella(), obtener_tablero_nuevo().

3.3. Mejora 3: Panel de elementos definido por el jugador (10 pts)

Descripción

Tercer tipo de elementos "Definido por jugador" además de números y letras. El jugador define 9 elementos distintos de máximo 2 caracteres en Configurar.

Implementación

- Radiobutton **"Definido por jugador"** (valor "custom") en sección Elementos de Configurar.
- Frame con 9 Entry en grilla 3×3, prellenados con valores guardados.
- Validaciones: 1-2 caracteres por elemento, todos distintos, sin vacíos.
- Método **_valor_a_texto(valor)**: centraliza conversión int 1-9 → texto visible.

Cambios en el programa

cargar_configuracion() agrega "elementos_custom": []. Nuevo método _valor_a_texto(). Múltiples métodos actualizados para usar _valor_a_texto(). abrir_configuracion() agrega Radiobutton y 9 Entry.

3.4. Mejora 4: Juegos multinivel (10 pts)

Descripción

Modo donde el jugador avanza automáticamente: Fácil → Intermedio → Difícil → Fácil. Al completar cada nivel se registra en el ABB correspondiente.

Implementación

- Radiobutton **"Multinivel"** en sección Nivel de Configurar.
- **self.nivel_actual_multinivel**: rastrea el nivel activo en el ciclo.
- **_completar_nivel_multinivel()**: inserta en ABB, calcula siguiente con índice modular, genera tablero.
- Timer multinivel: tiempos H/M/S independientes por nivel en config["timer_multinivel"].
- nivel_actual_multinivel guardado/cargado en JSON para continuar el ciclo.

Cambios en el programa

SudokuApp.__init__() agrega nivel_actual_multinivel="facil". click_casilla() bifurca entre normal y multinivel. Nuevo método _completar_nivel_multinivel(). abrir_configuracion() agrega Radiobutton y tabla timer.

3.5. Mejora 5: Cambios para el manejo del Top X con ABB (40 pts)

Descripción

Se reemplazó JSON de bitácora y ordenamiento burbuja por ABB con POO. Inserción y recorrido recursivos. ABB persistido con pickle.

Implementación

- • Bosque de 3 ABB globales: `abb_facil`, `abb_intermedio`, `abb_dificil`, cargados del PKL al iniciar.
- • `_insertar()` recursivo: menor→izquierda, mayor/igual→derecha.
- • `_en_orden()` recursivo: izquierdo→nodo→derecho, acumula strings de `get_partida()`.
- • `cargar_abbs()/guardar_abbs()`: `pickle.load()/pickle.dump()` con modo `"rb"/"wb"`.
- • `ver_top()` usa `recorrer_arbol()` en lugar de ordenamiento por burbuja.

Cambios en el programa

Import `pickle`. ARCHIVO_BITACORA cambia de `.json` a `.pkl`. Clases `Partida`, `NodoABB`, `ABB`. Funciones `cargar_abbs()`, `guardar_abbs()`, `insertar_en_abb()`. `click_casilla()` usa `insertar_en_abb()`.

3.6. Mejora 6: Funcionalidad propia — Botón PISTA (10 pts)

Descripción

Botón PISTA (naranja) que resuelve el tablero con backtracking y muestra la primera celda vacía con su valor correcto, resaltada en amarillo.

Implementación

- **_resolver_puzzle(tablero)**: backtracking que resuelve in-place. True si tiene solución.
- **mostrar_pista()** copia tablero, lo resuelve, busca primera celda vacía y muestra valor correcto.
- Celda en amarillo antes del messagebox, vuelve a blanco al cerrarlo.
- Usa **_valor_a_texto()** para formato correcto (número, letra o custom).
- Si irresoluble: "NO HAY PISTAS DISPONIBLES".

Cambios en el programa

Nueva función **_resolver_puzzle()**. Nuevo método **mostrar_pista()**. Botón "PISTA" (color naranja) agregado en **menu_botones** de **construir_botones()**, en la fila de menú junto a CONFIGURAR, AYUDA, ACERCA DE y SALIR. Al ejecutar el programa es inmediatamente visible en la tercera fila de botones, identificable por su color naranja igual al de CONFIGURAR.

4. Conclusiones

4.1. Problemas encontrados y soluciones

Problema: Doble tick del cronómetro al cambiar de nivel en multinivel

Descripción: Al detener con `cronometro_activo=False`, el callback pendiente en `root.after()` seguía activo. Si antes del siguiente segundo se reiniciaba el reloj, había doble ejecución.

Solución: Se introdujo `self._clock_handle`. Antes de iniciar nuevo ciclo se llama `after_cancel(self._clock_handle)`.

Problema: Pista mostraba primer candidato válido en lugar del valor correcto

Descripción: La implementación inicial buscaba el primer número 1-9 que pasara validaciones. En celdas con múltiples candidatos podía sugerir un valor incorrecto.

Solución: Se implementó `_resolver_puzzle()` que resuelve el tablero completo. La pista usa `solucion[fila][col]`.

Problema: Timer multinivel con 0/0/0 expiraba instantáneamente

Descripción: Si el usuario dejaba el tiempo de un sub-nivel en 0h 0m 0s, el timer ya estaba en cero al iniciar.

Solución: Validación en `guardar()`: cada sub-nivel debe tener al menos un valor > 0 con reloj Timer.

Problema: Label del nivel se corrompía al guardar configuración en partida multinivel

Descripción: Al guardar con juego multinivel en "intermedio", el label cambiaba de "multinivel (intermedio)" a "multinivel".

Solución: En `guardar()`, si el nivel es multinivel y hay juego activo, el label muestra el sub-nivel actual.

4.2. Aprendizajes obtenidos

- ABB con POO: el recorrido en-orden produce resultados ordenados automáticamente, sin algoritmo externo.
- Pickle: serializa objetos Python completos (instancias de ABB) sin conversión manual a texto.
- Git/GitHub: commits atómicos con mensajes descriptivos mantienen historial claro y facilitan corrección.
- SMTP: la seguridad incluye gestión correcta de credenciales del remitente (App Password de Google).
- Handles en `root.after()`: `cronometro_activo=False` no cancela el callback; se requiere `after_cancel()` explícito.

5. Estadística de tiempos

Actividad Realizada	Horas
Análisis del código base del Programa 2	2
Diseño de la arquitectura de mejoras	2
Investigación: control de versiones, Git, GitHub	3
Configuración del repositorio GitHub y commits iniciales	1
Implementación Mejora 2: generador en tiempo real	3
Implementación Mejora 1: sistema de cuentas y login SMTP	4
Implementación Mejora 5: clases ABB + pickle	4
Implementación Mejora 3: elementos definidos por jugador	2
Implementación Mejora 4: juegos multinivel	3
Implementación Mejora 6: botón PISTA	2
Corrección de bugs (multinivel, pista, reloj, label)	3
Documentación interna del código	2
Pruebas funcionales generales	3
Elaboración del manual de usuario	3
Elaboración de la documentación del proyecto	4
TOTAL	41

6. Lista de revisión del proyecto

Concepto	Pts	%	Análisis de resultados
Esquema de cuentas para acceder a la aplicación	20	100%	Login con correo, SHA-256, Gmail SMTP, nombre del archivo.
Generar partidas automáticamente	10	100%	Backtracking + unicidad. Anti-repetición deque(maxlen=50).
Panel de elementos	10	100%	9 elementos custom con validaciones. _valor_a_texto() centralizado.
Juegos multinivel	10	100%	Ciclo F→I→D→F. Timer por sub-nivel. nivel guardado/cargado.
Cambios para el manejo del Top X (ABB)	40	100%	Clases Partida, NodoABB, ABB recursivos. 3 ABB. Pickle.
Funcionalidad propia: botón PISTA	10	100%	Backtracking completo. Celda amarilla con valor correcto.
TOTAL	100	100%	Todas las mejoras implementadas y probadas.

Funcionalidades adicionales:

- •_resolver_puzzle() reutilizable para verificar estado del tablero.
- •self._clock_handle para cancelación segura del cronómetro.
- •Migración automática del esquema JSON entre versiones.